Programmierbare Bausteine

Vorlesung 06

Dr.-Ing. Christian Hakert / Dr. rer. nat. Mario Günzel

TU Dortmund, Fakultät für Informatik, Lehrstuhl 12

by Decker

Übersicht

1. Organisatorisches ✓
2. Einleitung ✓
3. Repräsentation von Daten ✓
4. Boolesche Funktionen und Schaltnetze ✓
5. Rechnerarithmetik ✓
6. Optimierung von Schaltnetzen ✓
7. **Programmierbare Bausteine**
8. Synchrone Schaltwerke

Prorammierbare Bausteine

6. **Programmierbare Bausteine**
    1. **Einleitung**
    2. Grundbausteine
    3. Realisierung von Monomen und Polynomen
    4. PLA als Speicher
    5. Software PLAs

Einleitung

Realisierung von Schaltnetzen

Gedanken zur Anwendung

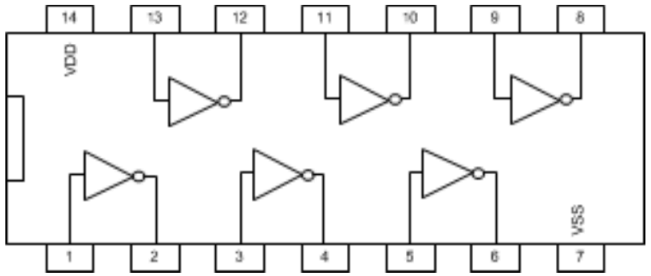
1. Problem
2. Boolesche Funktion
3. Schaltnetz-Entwurf
4. Schaltnetz-Realisierung

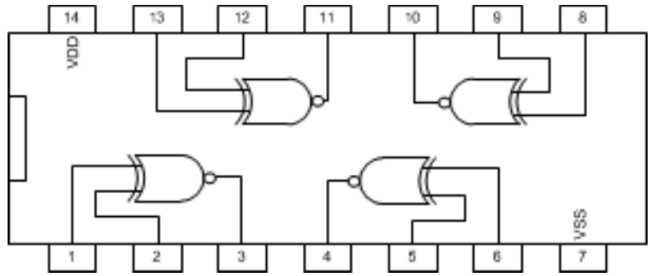
Realisierungen

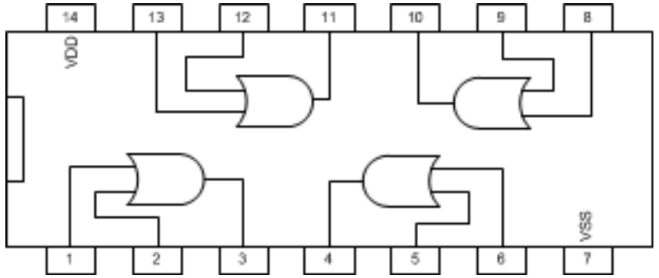
- Hoch-integrierte Schaltung
  Teuer Lohnt sich nur bei großen Stückzahlen
- Direkte Umsetzung mit Gattern
  Umständlich

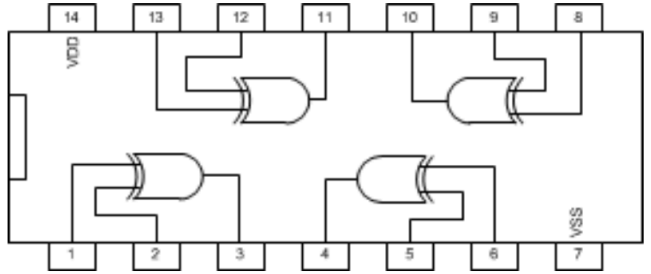
Einleitung

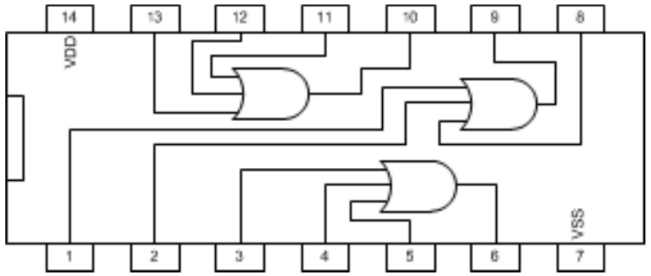
Realisierung mit Gattern

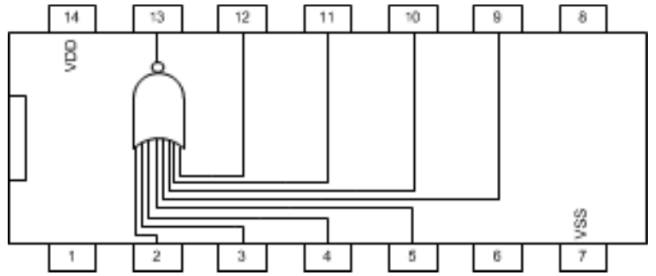






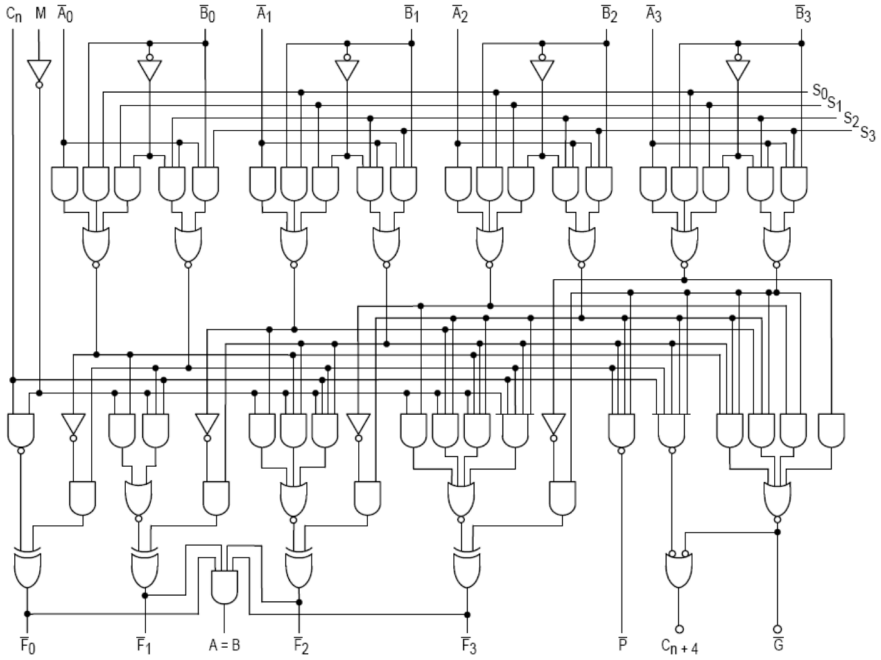




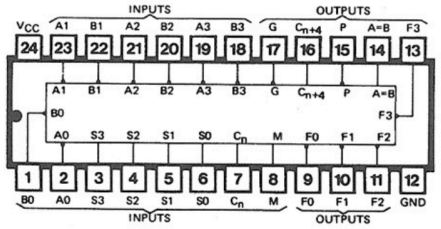


Einleitung

Realisierung mit Gattern



Quelle: en.wikibooks.org/wiki/Microprocessor_Design/ALU, Autor: Poil



$S3$	$S2$	$S1$	$S0$	$M=1$	$M=0$ $CIN=1$	$M=0$ $CIN=0$
0	0	0	0	$\overline{A}$	A	$A+1$
0	0	0	1	$\overline{A\|B}$	$A\|B$	$A\|B+1$
0	0	1	0	$\overline{A}B$	$A\|\overline{B}$	$A\|\overline{B}+1$
0	0	1	1	0	-1	0
0	1	0	0	$\overline{AB}$	$A+A\overline{B}$	$A+A\overline{B}+1$
0	1	0	1	$\overline{B}$	$(A\|B)+A\overline{B}$	$(A\|B)+A\overline{B}+1$
0	1	1	0	$A\oplus B$	$A-B-1$	$A-B$
0	1	1	1	$A\overline{B}$	$A\overline{B}-1$	$A\overline{B}$
1	0	0	0	$\overline{A}\|B$	$A+AB$	$A+AB+1$
1	0	0	1	$\overline{A\oplus B}$	$A+B$	$A+B+1$
1	0	1	0	B	$(A\|\overline{B})+AB$	$(A\|\overline{B})+AB+1$
1	0	1	1	AB	$AB-1$	AB
1	1	0	0	1	$2*A$	$2*A+1$
1	1	0	1	$A\|\overline{B}$	$(A\|B)+A$	$(A\|B)+A+1$
1	1	1	0	$A\|B$	$(A\|\overline{B})+A$	$(A\|\overline{B})+A+1$
1	1	1	1	A	$A-1$	A

Einleitung

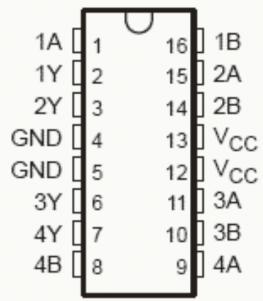
Realisierung mit Gattern

74AC11008
QUADRUPLE 2-INPUT POSITIVE-AND GATE

SCAS014C – AUGUST 1987 – REVISED APRIL 1996

- **Flow-Through Architecture Optimizes PCB Layout**
- **Center-Pin V_{CC} and GND Configurations Minimize High-Speed Switching Noise**
- ***EPIC*™ (Enhanced-Performance Implanted CMOS) 1-μm Process**
- **500-mA Typical Latch-Up Immunity at 125°C**
- **Package Options Include Plastic Small-Outline (D) and Thin Shrink Small-Outline (PW) Packages, and Standard Plastic 300-mil DIPs (N)**

D, N, OR PW PACKAGE
(TOP VIEW)



description

This device contains four independent 2-input AND gates. It performs the Boolean function $Y = A \bullet B$ or $Y = \overline{\overline{A} + \overline{B}}$ in positive logic.

The 74AC11008 is characterized for operation from –40°C to 85°C.

FUNCTION TABLE
(each gate)

INPUTS		OUTPUT
A	B	Y
H	H	H
L	X	L
X	L	L

Einleitung

Alternative Realisierung durch

- Massenhaft produzierte
- … darum **preisgünstige**
- … nach der Fertigstellung in ihrer Funktion noch beeinflussbare
- … **funktional vollständige**
- … also **universelle** Standardbausteine

Programmable Logic Array (PLA)

Varianten	PAL, PROM, FPGA (Zum Teil eingeschränkte Funktionalität)

Einleitung

Programmable Logic Array (PLA) Datenblatt von Lattice

Lattice Semiconductor Corporation

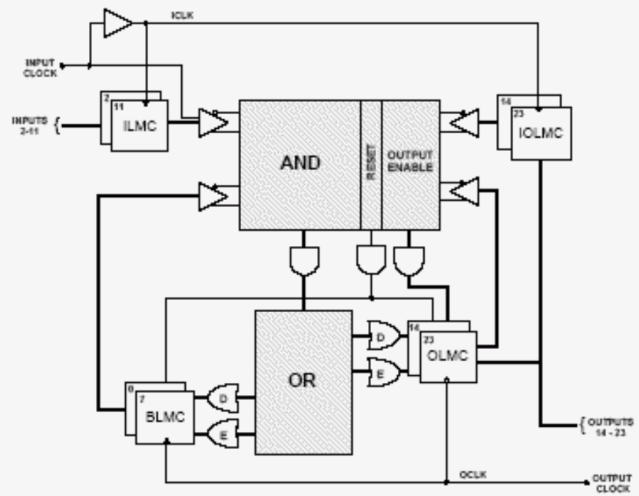
GAL6001

High Performance E²CMOS FPLA Generic Array Logic™

Features

- HIGH PERFORMANCE E²CMOS® TECHNOLOGY
  - 30ns Maximum Propagation Delay
  - 27MHz Maximum Frequency
  - 12ns Maximum Clock to Output Delay
  - TTL Compatible 16mA Outputs
  - UltraMOS® Advanced CMOS Technology
- LOW POWER CMOS
  - 90mA Typical Icc
- E² CELL TECHNOLOGY
  - Reconfigurable Logic
  - Reprogrammable Cells
  - 100% Tested/100% Yields
  - High Speed Electrical Erasure (<100ms)
  - 20 Year Data Retention
- UNPRECEDENTED FUNCTIONAL DENSITY
  - 78 x 64 x 36 FPLA Architecture
  - 10 Output Logic Macrocells
  - 8 Buried Logic Macrocells
  - 20 Input and I/O Logic Macrocells
- HIGH-LEVEL DESIGN FLEXIBILITY
  - Asynchronous or Synchronous Clocking
  - Separate State Register and Input Clock Pins
  - Functional Superset of Existing 24-pin PAL® and FPLA Devices

Functional Block Diagram



Macrocell Names

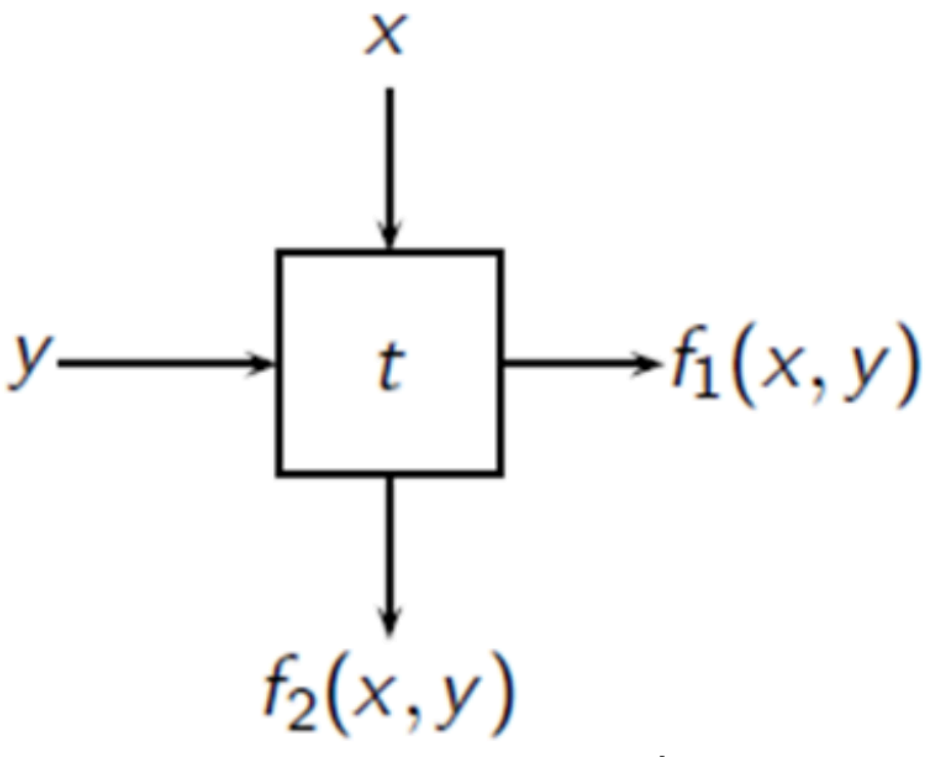
ILMC	INPUT LOGIC MACROCELL
IOLMC	I/O LOGIC MACROCELL
BLMC	BURIED LOGIC MACROCELL
OLMC	OUTPUT LOGIC MACROCELL

Programmierbare Bausteine

6. **Programmierbare Bausteine**
    1. Einleitung ✓
    2. **Grundbausteine**
    3. Realisierung von Monomen und Polynomen
    4. PLA als Speicher
    5. Software PLAs

Grundbausteine

PLA-Grundbausteine



Name	Typ	$f_1(x,y)$	$f_2(x,y)$
Identer	0	y	x
Addierer	1	$x \vee y$	x
Multiplizierer	2	y	$x \wedge y$
Negat-Multiplizierer	3	y	$x \wedge \overline{y}$

Funktional vollständig?

Grundbausteine

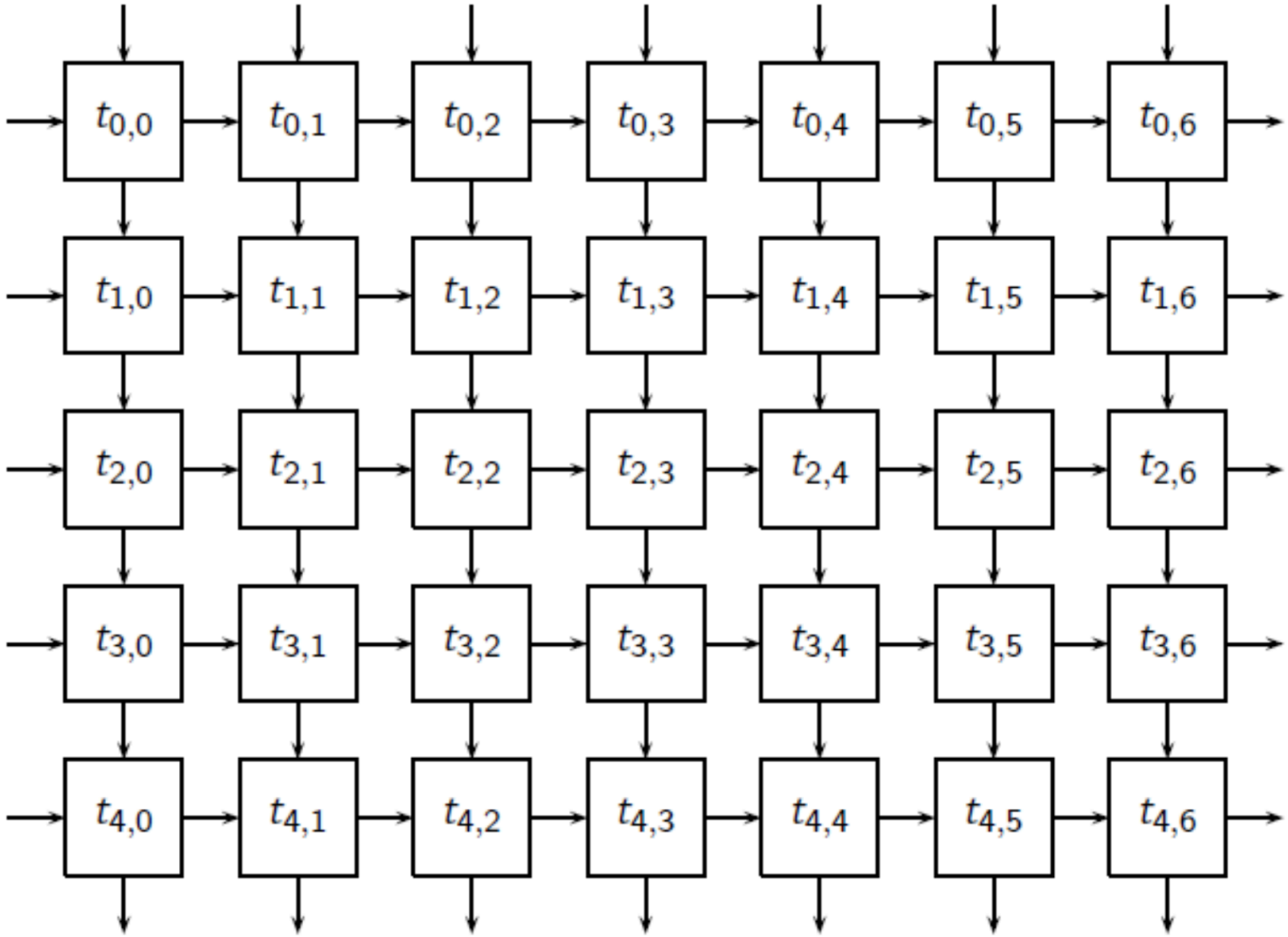
Name	Typ	$f_1(x, y)$	$f_2(x, y)$
Identer	0	y	x
Addierer	1	$x \vee y$	x
Multiplizierer	2	y	$x \wedge y$
Negat-Multiplizierer	3	y	$x \wedge \overline{y}$

Funktional vollständig!

- UND-Verknüpfung: Typ 2, $f_2(x, y) = x \wedge y$
- ODER-Verknüpfung: Typ 1, $f_1(x, y) = x \vee y$
- Negation: Typ 3, $f_2(1, y) = 1 \wedge \overline{y} = \overline{y}$

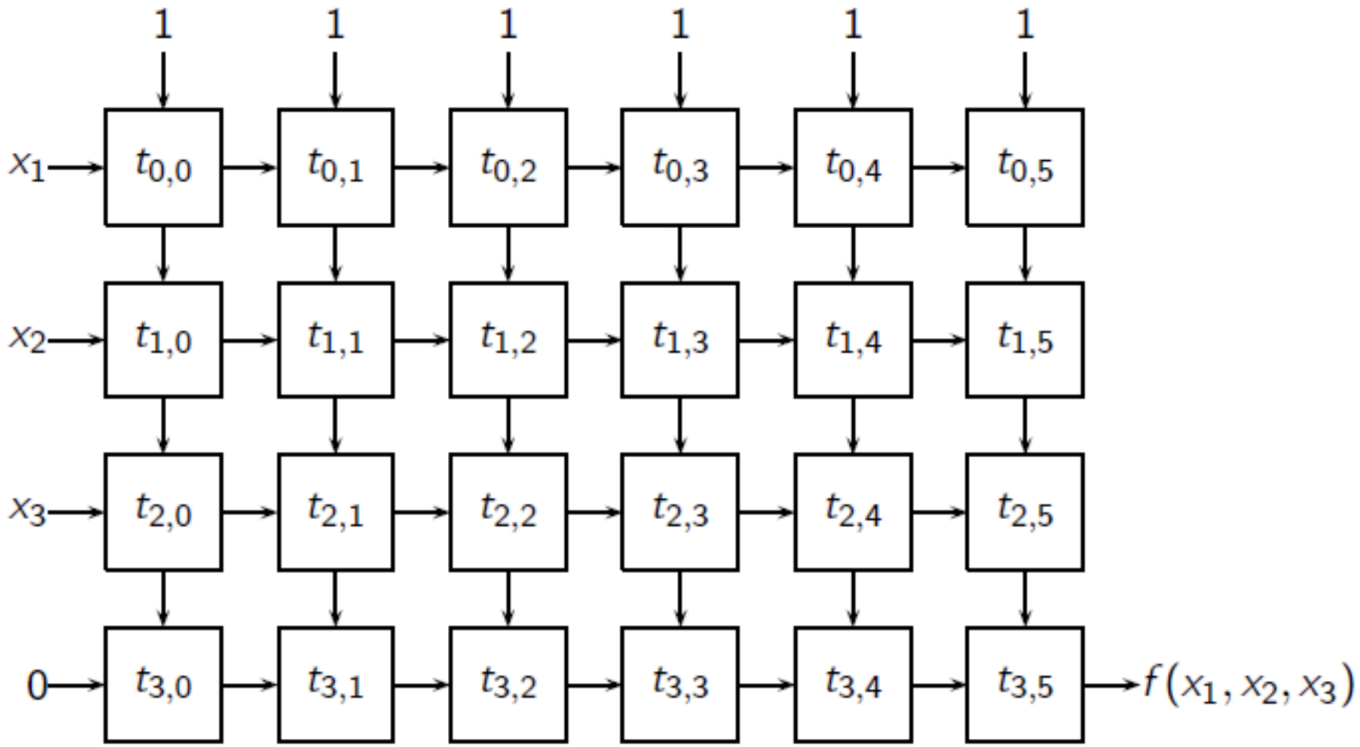
Grundbausteine

Aufbau eines PLA



Grundbausteine

PLA für $f : \{0,1\}^3 \to \{0,1\}$



Wie wählt man die Bausteintypen?

Programmierbare Bausteine

6. **Programmierbare Bausteine**
    1. Einleitung ✓
    2. Grundbausteine ✓
    3. **Realisierung von Monomen und Polynomen**
    4. PLA als Speicher
    5. Software PLAs

Realisierung von Monomen und Polynomen

Auswahl der Bausteine

Feststellung Jede Funktion als **Polynom** darstellbar

Erinnerung Polynom = Disjunktion einiger Monome

Erster Schritt

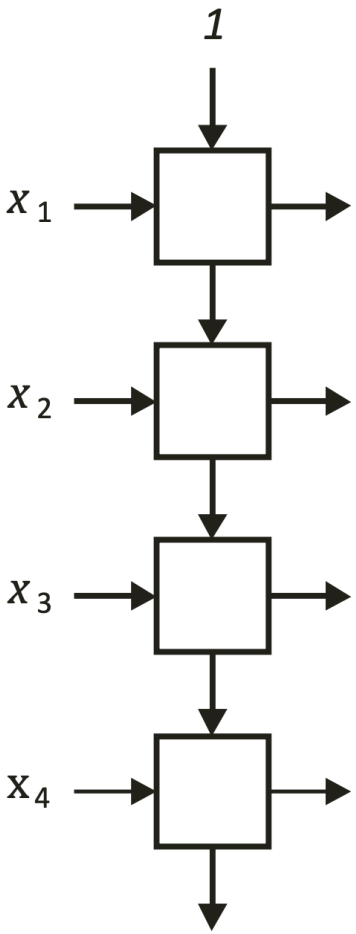
- Wie realisieren wir Monome?
- Exemplarisch am Beispiel $x_1 \overline{x_2}\, x_4$

Realisierung von Monomen und Polynomen

Realisierung von Monomen

Beispiel: Monom $x_1 \overline{x_2}\, x_4$

Name	**Typ**	$f_r(o, l)$	$f_u(o, l)$
Identer	0	l	o
Addierer	1	$l \vee o$	o
Multiplizierer	2	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$

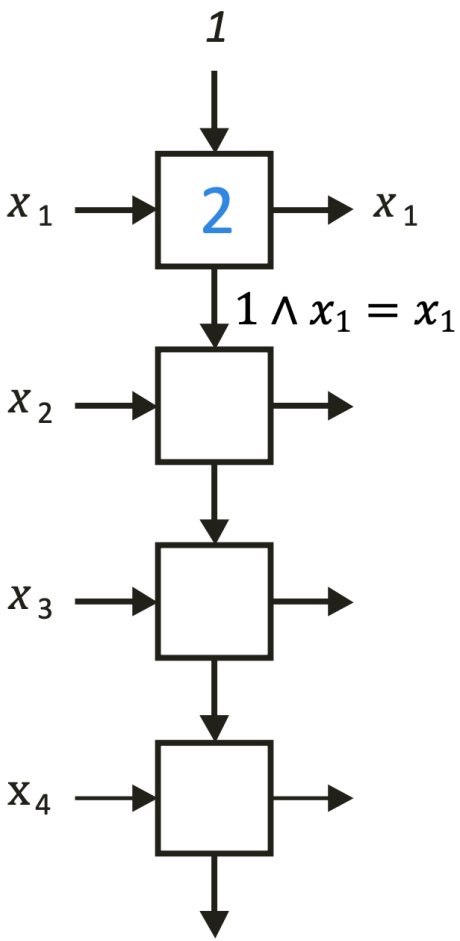

1
x_1
x_2
x_3
x_4

Realisierung von Monomen und Polynomen

Realisierung von Monomen

Beispiel: Monom $x_1 \overline{x_2}\, x_4$

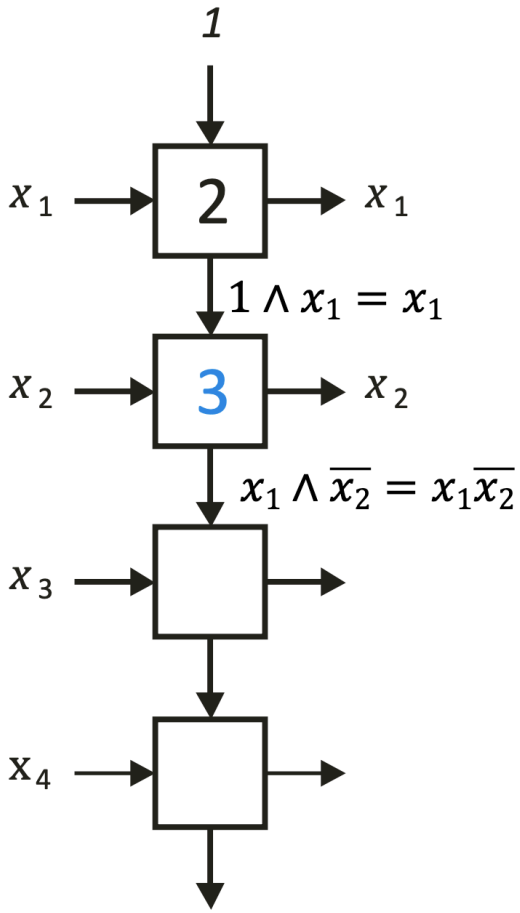
Name	**Typ**	$f_r(o, l)$	$f_u(o, l)$
Identer	0	l	o
Addierer	1	$l \vee o$	o
Multiplizierer	**2**	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$


1
2
x_1
x_1
$1 \wedge x_1 = x_1$
x_2
x_3
x_4

Realisierung von Monomen und Polynomen

Realisierung von Monomen

Beispiel: Monom $x_1 \overline{x_2}\, x_4$



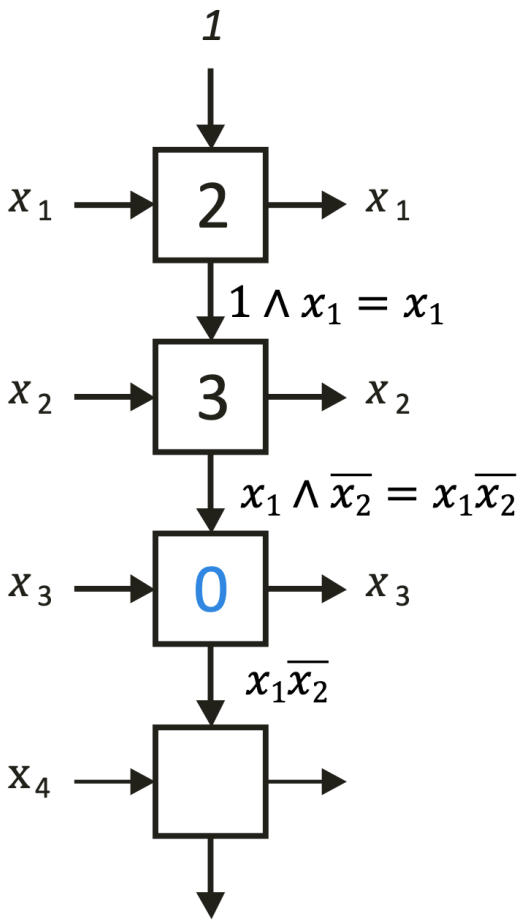
Name	**Typ**	$f_r(o,l)$	$f_u(o,l)$
Identer	0	l	o
Addierer	1	$l \vee o$	o
Multiplizierer	2	l	$o \wedge l$
Negat-Multiplizierer	**3**	l	$o \wedge \bar{l}$

Realisierung von Monomen und Polynomen

Realisierung von Monomen

Beispiel: Monom $x_1 \overline{x_2}\, x_4$

Name	Typ	$f_r(o, l)$	$f_u(o, l)$
Identer	**0**	l	o
Addierer	1	$l \vee o$	o
Multiplizierer	2	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$

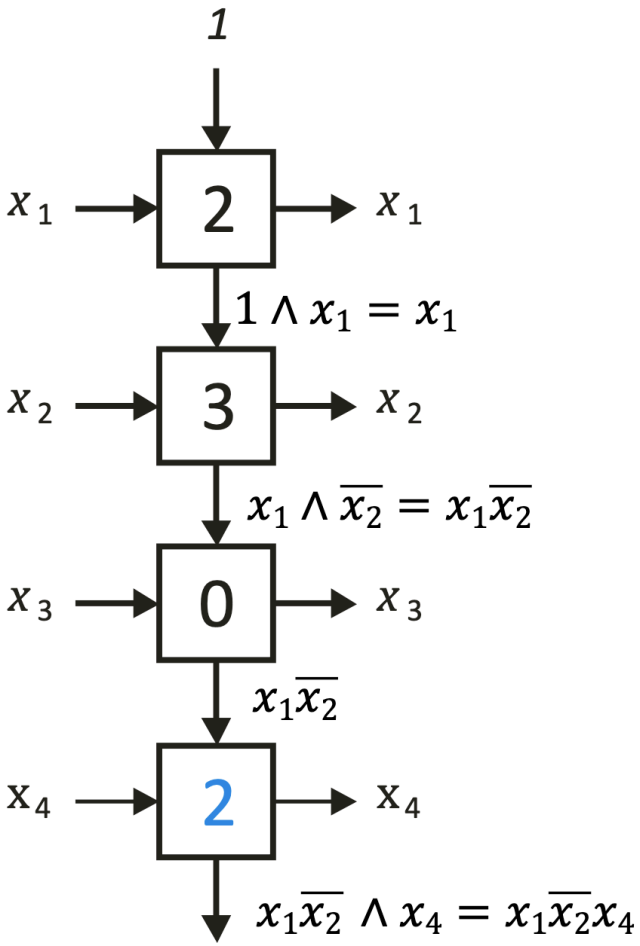

1
x_1
2
x_1
$1 \wedge x_1 = x_1$
x_2
3
x_2
$x_1 \wedge \overline{x_2} = x_1\overline{x_2}$
x_3
0
x_3
$x_1\overline{x_2}$
x_4

Realisierung von Monomen und Polynomen

Realisierung von Monomen

Beispiel: Monom $x_1 \overline{x_2}\, x_4$

Name	**Typ**	$f_r(o, l)$	$f_u(o, l)$
Identer	0	l	o
Addierer	1	$l \vee o$	o
Multiplizierer	**2**	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$


1
x_1
2
x_1
$1 \wedge x_1 = x_1$
x_2
3
x_2
$x_1 \wedge \overline{x_2} = x_1\overline{x_2}$
x_3
0
x_3
$x_1\overline{x_2}$
x_4
2
x_4
$x_1\overline{x_2} \wedge x_4 = x_1\overline{x_2}x_4$

Realisierung von Monomen und Polynomen

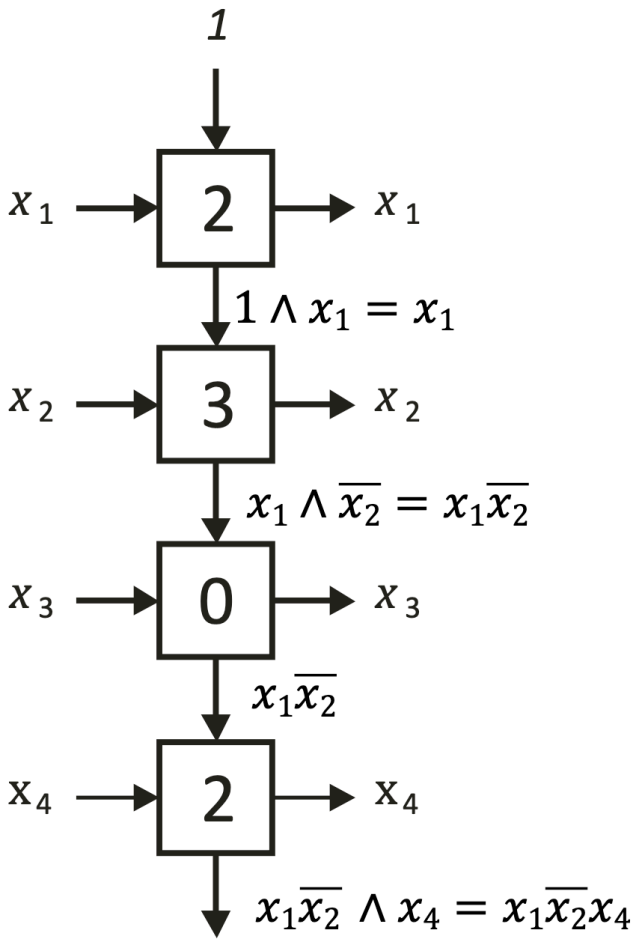
Realisierung von Monomen

Beispiel: Monom $x_1 \overline{x_2}\, x_4$

Name	Typ	$f_r(o, l)$	$f_u(o, l)$
Identer	0	l	o
Addierer	1	$l \vee o$	o
Multiplizierer	2	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$

Monomrealisierung

- Falls Variable fehlt Typ 0
- Falls x_i vorkommt Typ 2
- Falls $\overline{x_i}$ vorkommt Typ 3


1
x_1
2
x_1
$1 \wedge x_1 = x_1$
x_2
3
x_2
$x_1 \wedge \overline{x_2} = x_1\overline{x_2}$
x_3
0
x_3
$x_1\overline{x_2}$
x_4
2
x_4
$x_1\overline{x_2} \wedge x_4 = x_1\overline{x_2}x_4$

Realisierung von Monomen und Polynomen

Realisierung von Polynomen

Beobachtung

- Für $f : \{0,1\}^n \to \{0,1\}$
- **k** verschiedene **Monome** $m_1, m_2, \ldots, m_k$
- In *n* Zeilen und *k* Spalten realisierbar

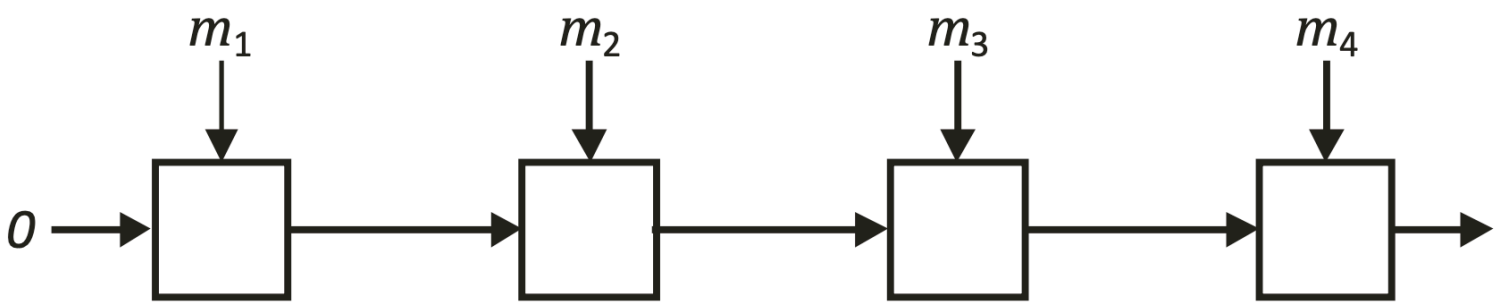
Wie können wir f realisieren, z.B. $f = m_1 \vee m_2 \vee m_4$?

Realisierung von Monomen und Polynomen

Realisierung von Polynomen

Beispiel: Polynom $f = m_1 \vee m_2 \vee m_4$

Name	**Typ**	$f_r(o, l)$	$f_u(o, l)$
Identer	0	l	o
Addierer	1	$l \vee o$	o
Multiplizierer	2	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$

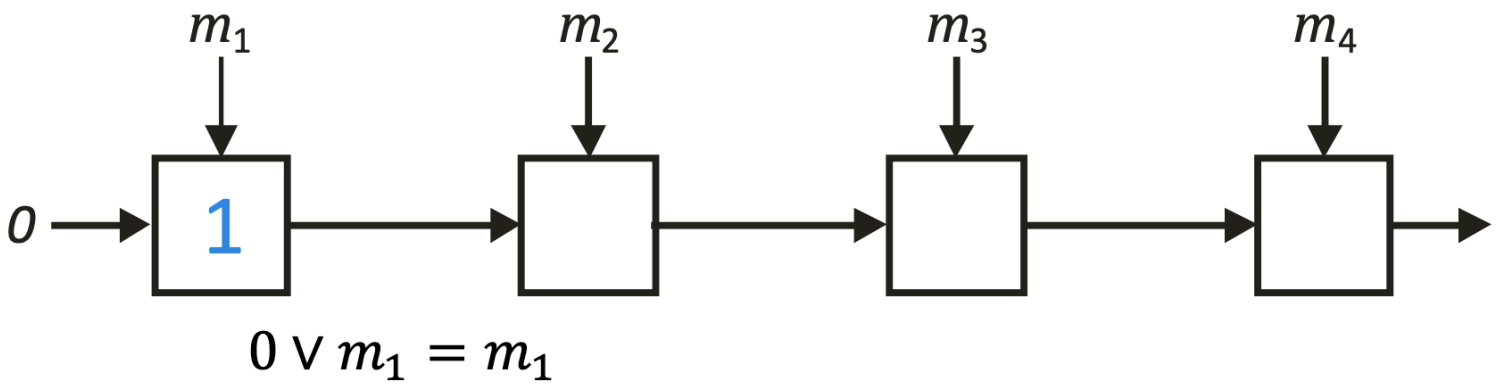

m_1
m_2
m_3
m_4
0

Realisierung von Monomen und Polynomen

Realisierung von Polynomen

Beispiel: Polynom $f = m_1 \vee m_2 \vee m_4$

Name	**Typ**	$f_r(o, l)$	$f_u(o, l)$
Identer	0	l	o
Addierer	**1**	$l \vee o$	o
Multiplizierer	2	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$



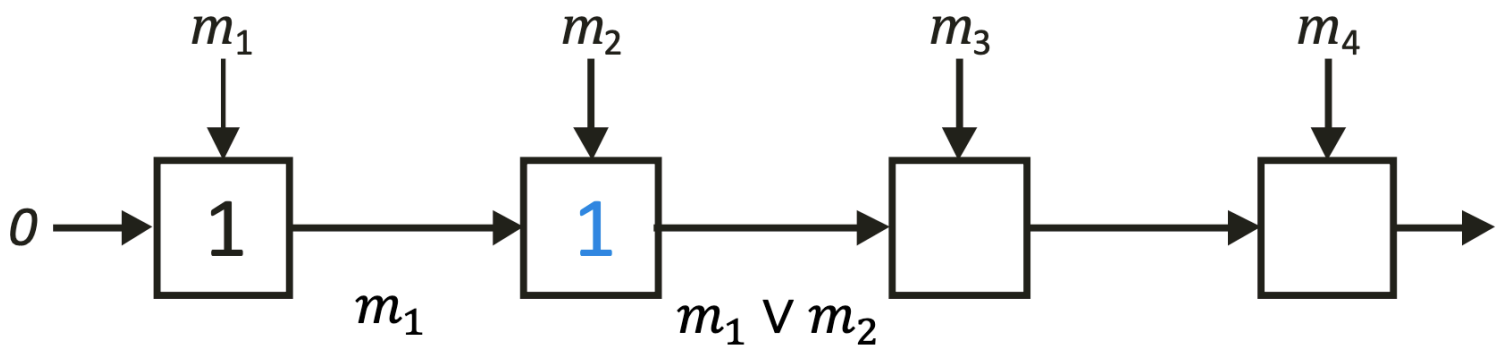
$0 \vee m_1 = m_1$

Realisierung von Monomen und Polynomen

Realisierung von Polynomen

Beispiel: Polynom $f = m_1 \vee m_2 \vee m_4$

Name	**Typ**	$f_r(o, l)$	$f_u(o, l)$
Identer	0	l	o
Addierer	**1**	$l \vee o$	o
Multiplizierer	2	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$

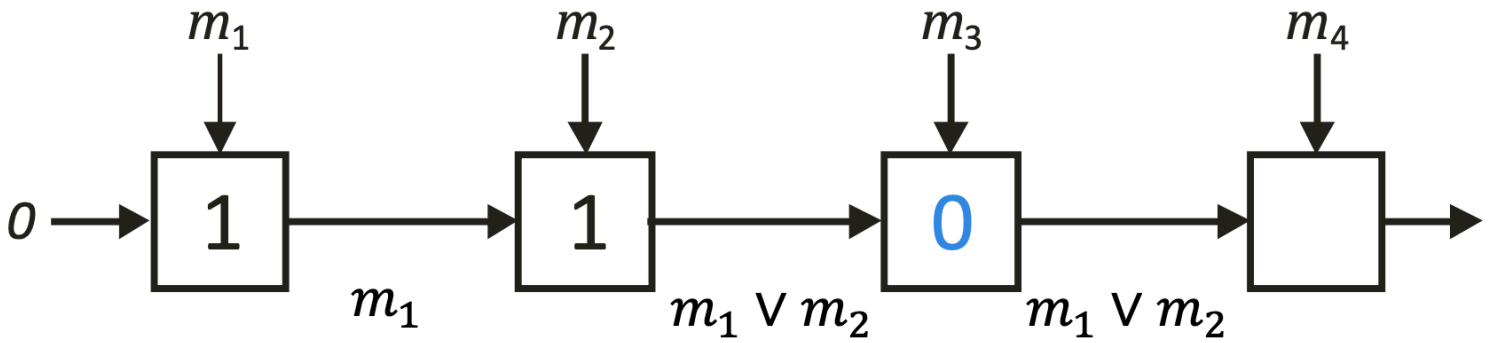

m_1
m_2
m_3
m_4
0
1
1
m_1
$m_1 \vee m_2$

Realisierung von Monomen und Polynomen

Realisierung von Polynomen

Beispiel: Polynom $f = m_1 \vee m_2 \vee m_4$

Name	**Typ**	$f_r(o, l)$	$f_u(o, l)$
Identer	**0**	l	o
Addierer	1	$l \vee o$	o
Multiplizierer	2	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$

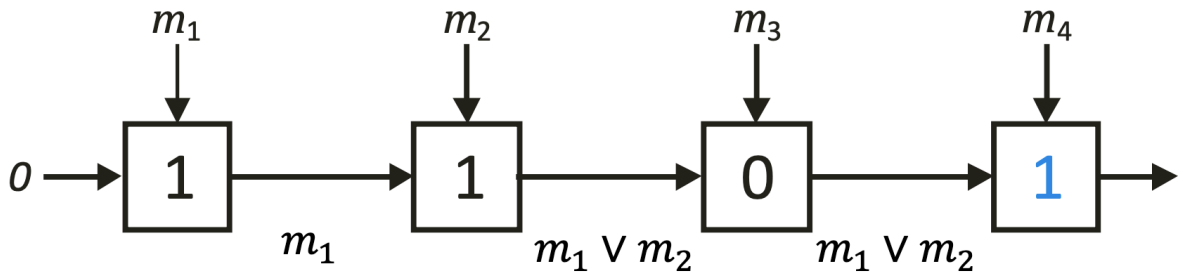

m_1
m_2
m_3
m_4
0
1
1
0
m_1
m_1 V m_2
m_1 V m_2

Realisierung von Monomen und Polynomen

Realisierung von Polynomen

Beispiel: Polynom $f = m_1 \vee m_2 \vee m_4$

Name	Typ	$f_r(o, l)$	$f_u(o, l)$
Identer	**0**	l	o
Addierer	1	$l \vee o$	o
Multiplizierer	2	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$

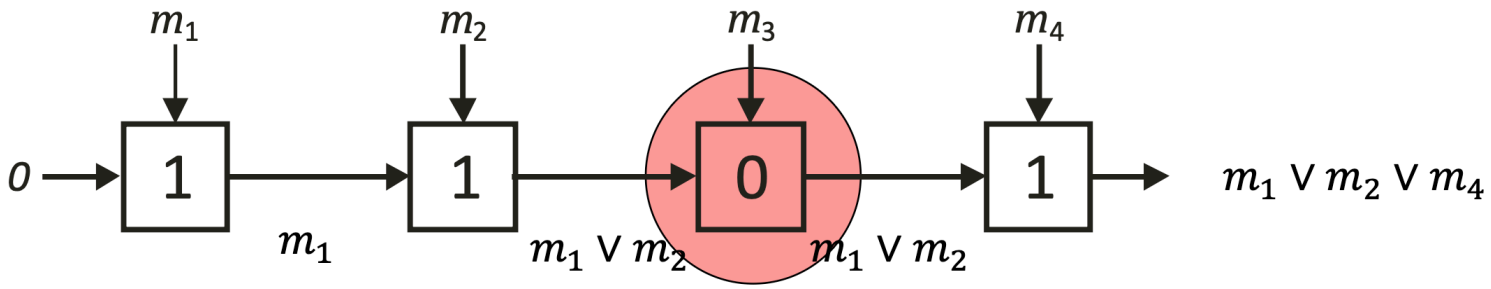


Realisierung von Monomen und Polynomen

Realisierung von Polynomen

Beispiel: Polynom $f = m_1 \vee m_2 \vee m_4$

Name	Typ	$f_r(o, l)$	$f_u(o, l)$
Identer	**0**	l	o
Addierer	1	$l \vee o$	o
Multiplizierer	2	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$



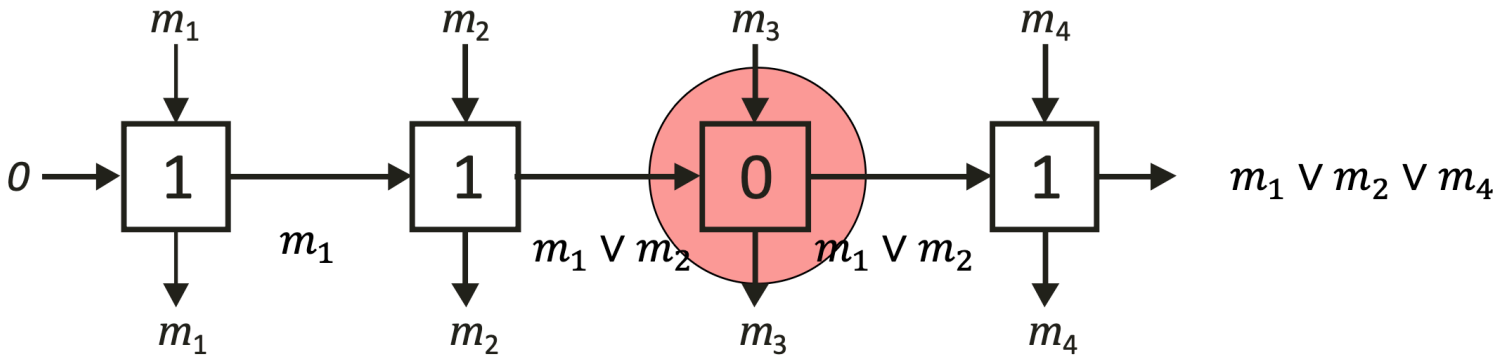
Warum ein Grundbaustein für das Weglassen eines Monoms?

Realisierung von Monomen und Polynomen

Realisierung von Polynomen

Beispiel: Polynom $f = m_1 \vee m_2 \vee m_4$

Name	**Typ**	$f_r(o, l)$	$f_u(o, l)$
Identer	**0**	l	o
Addierer	1	$l \vee o$	o
Multiplizierer	2	l	$o \wedge l$
Negat-Multiplizierer	3	l	$o \wedge \bar{l}$



Warum ein Grundbaustein für das Weglassen eines Monoms?

Realisierung von Monomen und Polynomen

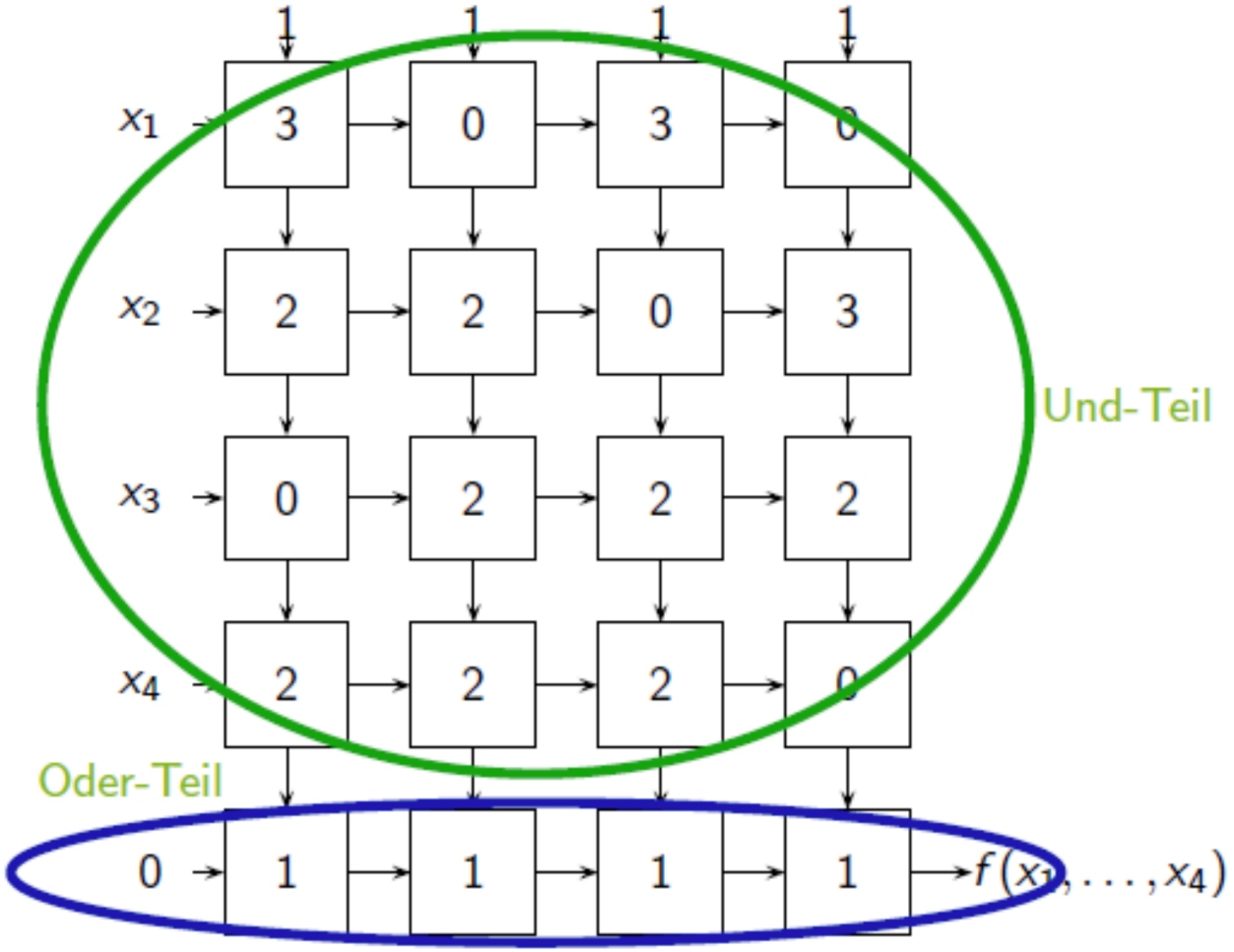
PLA: Ein konkretes Beispiel

Beispiel: $f(x_1, x_2, x_3, x_4) = \overline{x_1}\, x_2 x_4 \vee x_2 x_3 x_4 \vee \overline{x_1}\, x_3 x_4 \vee \overline{x_2}\, x_3$

Realisierung von Monomen und Polynomen

PLA: Ein konkretes Beispiel

Beispiel: $f(x_1, x_2, x_3, x_4) = \overline{x_1}\, x_2 x_4 \vee x_2 x_3 x_4 \vee \overline{x_1}\, x_3 x_4 \vee \overline{x_2}\, x_3$


1 1 1 1
x_1 3 0 3 0
x_2 2 2 0 3
x_3 0 2 2 2
x_4 2 2 2 0
Und-Teil
Oder-Teil
0 1 1 1 1
$f(x_1, \ldots, x_4)$

Realisierung von Monomen und Polynomen

Realisierung von Polynomen

Beobachtung 1

- Für $f : \{0,1\}^n \to \{0,1\}$
- *k* verschiedene **Monome** $m_1, m_2, \ldots, m_k$
- In *n + 1* Zeilen und *k* Spalten realisierbar

Beobachtung 2

- Wir können jede Funktion $f : \{0,1\}^n \to \{0,1\}^m$
- … für deren **Polynom** insgesamt *k* **Implikanten** ausreichen,
- … mit einem PLA mit *n + m* Zeilen und *k* Spalten realisieren

Quiz

Wenn ein Polynom mit einem PLA realisiert wird, wie groß ist das PLA für ein Polynom für $f : B^5 \rightarrow B^1$ mit 3 monomen?

A: 5x3

B: 6x3

C: 5x4

D: 6x4

E: kommt auf die maximale Länge der Monome an

Programmierbare Bausteine

6. **Programmierbare Bausteine**
    1. Einleitung ✓
    2. Grundbausteine ✓
    3. Realisierung von Monomen und Polynomen ✓
    4. **PLA als Speicher**
    5. Software PLAs

PLA als Speicher

PLA als ROM

Aufgabe: Speichere 2^n “Wörter” der Länge *m*. $w_0 = w_{0,0}w_{0,1}w_{0,2}\dots w_{0,m-1} \in \{0,1\}^m$ $w_0 = w_{1,0}w_{1,1}w_{1,2}\dots w_{1,m-1} \in \{1,1\}^m$ $w_0 = w_{2,0}w_{2,1}w_{2,2}\dots w_{2,m-1} \in \{2,1\}^m$

$$\begin{array}{ccc} . & . & . \\ . & . & . \\ . & . & . \end{array}$$

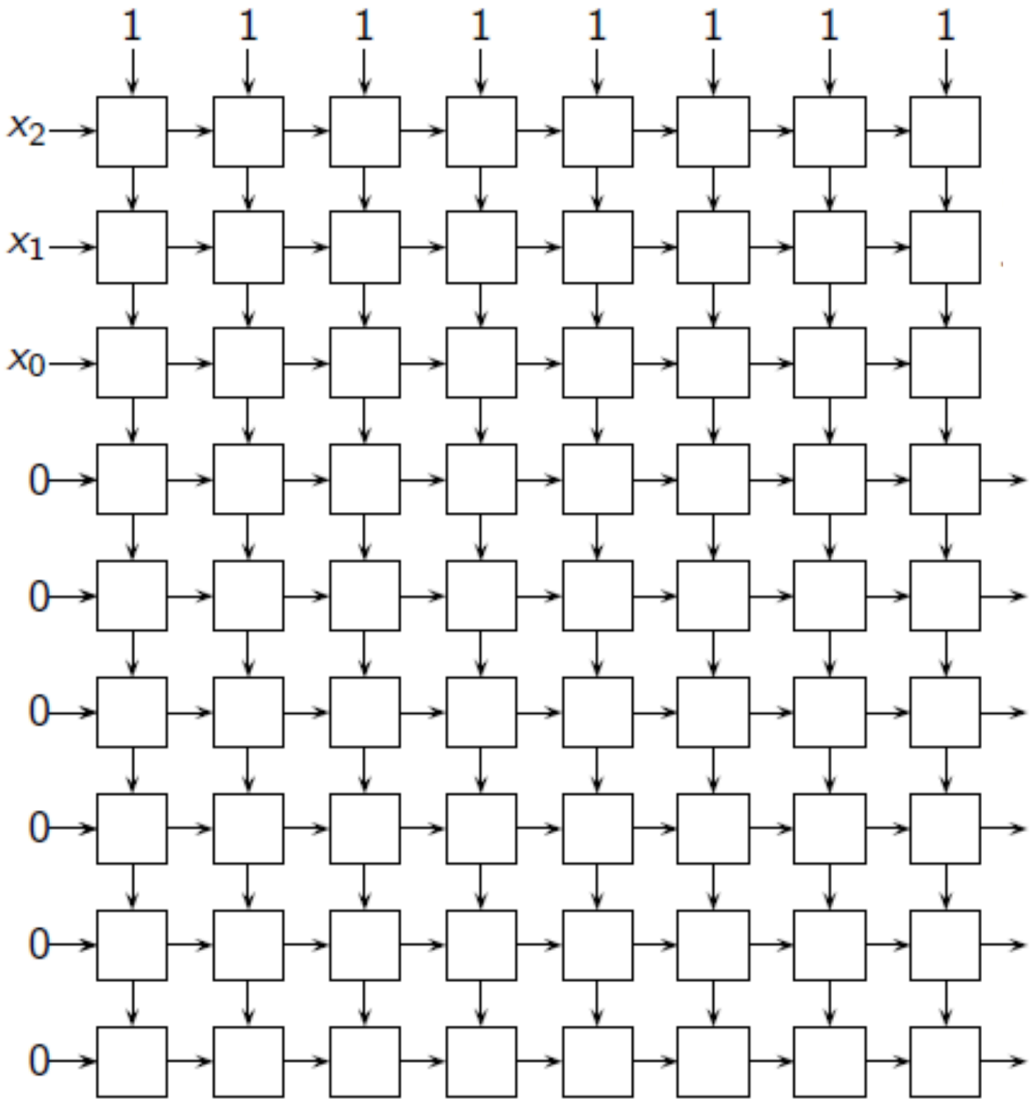
$w_{2^n-1} = w_{2^n-1,0}\ w_{2^n-1,1}\ w_{2^n-1,2} \dots w_{2^n-1,m-1} \in \{2,1\}^m$

Benutze **PLA** mit n + m Zeilen, 2^n Spalten

Beobachtung $m * 2^n$ Zellen für $m * 2^n$ zu speichernde Bits mindestens erforderlich

Adressierung Mit jeweils *n* Bits in *n* zusätzlichen Zeilen

PLA als Speicher



Beispiel

$n = 3, m = 6$

$2^3 = 8$ Wörter

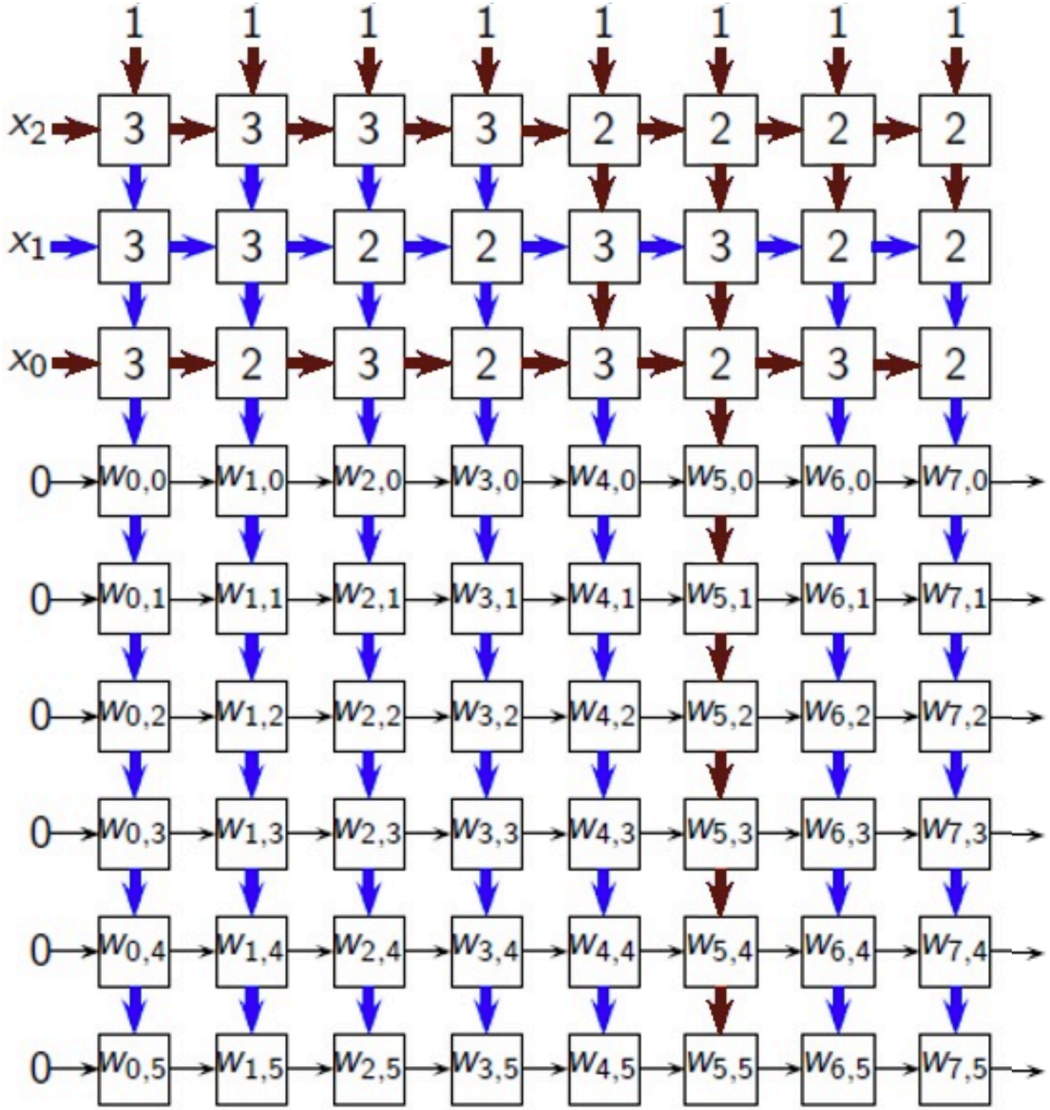
Jeweils 6 Bits

PLA als Speicher

PLA als Speicher

PLA als Speicher

PLA als Speicher



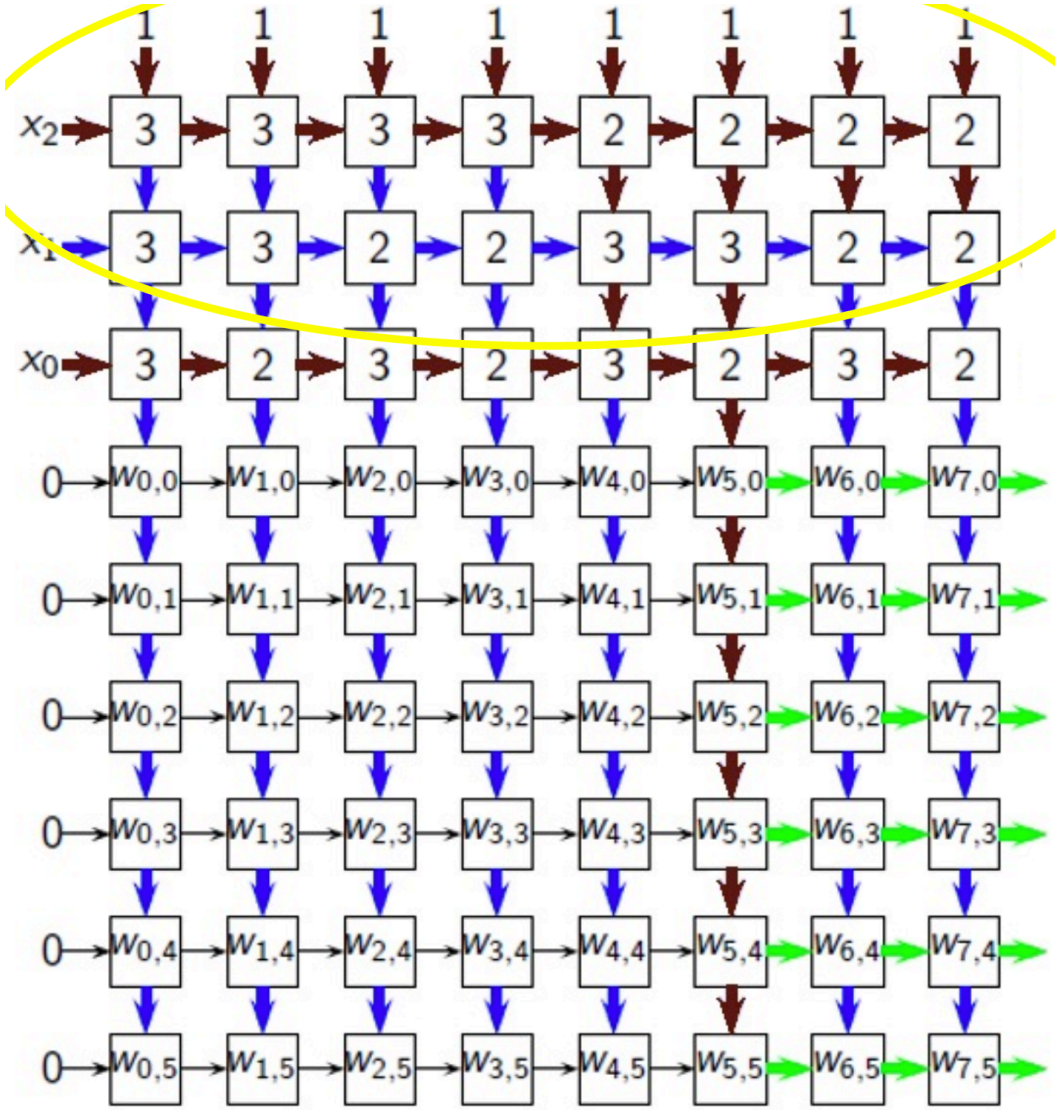
Beispiel

$n = 3, m = 6$
$2^3 = 8$ Wörter
Jeweils 6 Bits

Beispiel

Lies w_5
$5 = (101)_2$
$x_2 = 1$
$x_1 = 0$
$x_0 = 1$

PLA als Speicher



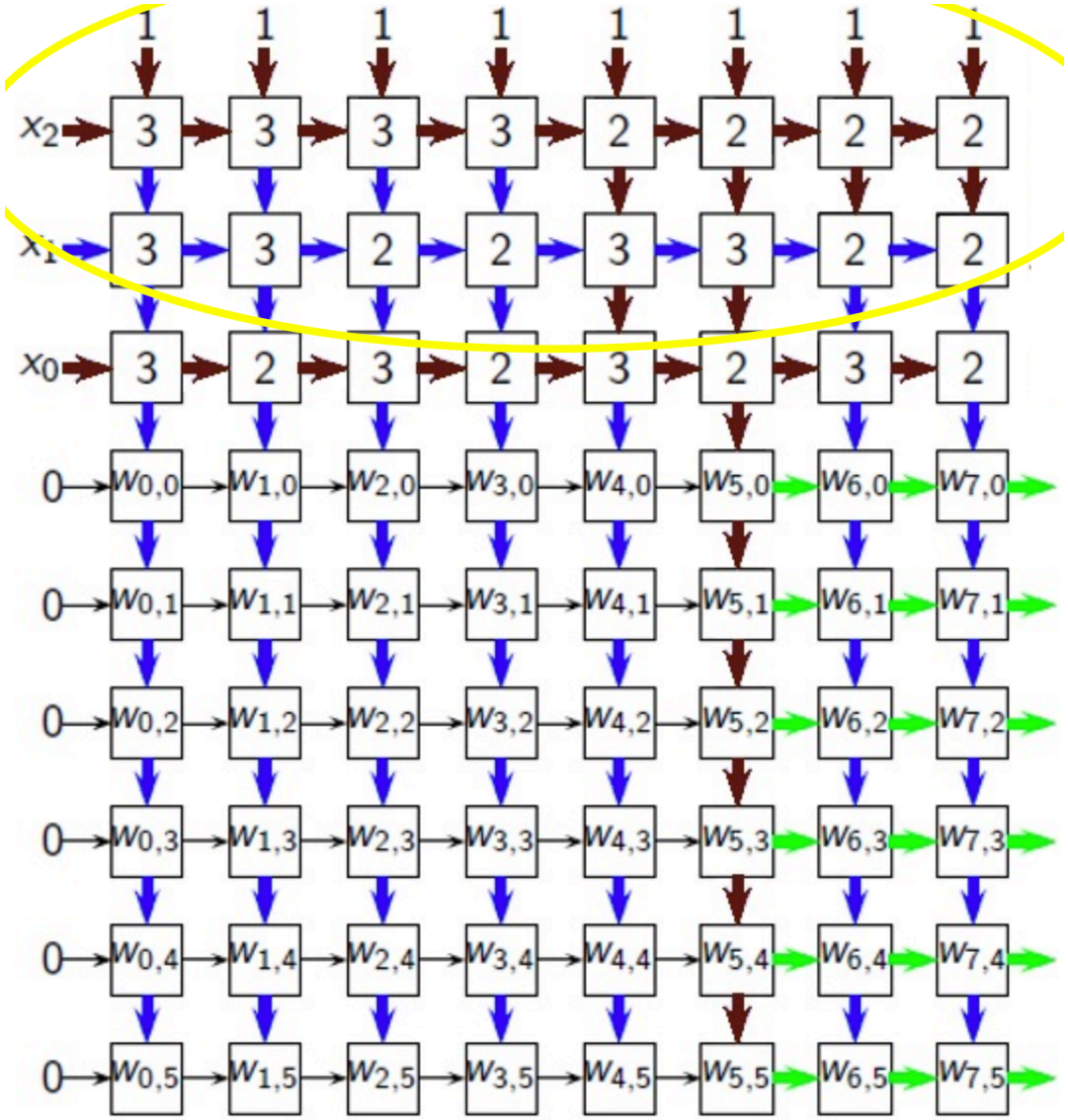
Beispiel

$n = 3, m = 6$
$2^3 = 8$ Wörter
Jeweils 6 Bits

Beispiel

Lies w_5
$5 = (101)_2$
$x_2 = 1$
$x_1 = 0$
$x_0 = 1$

PLA als Speicher



Beispiel

$n = 3, m = 6$
$2^3 = 8$ Wörter
Jeweils 6 Bits

Beispiel

Lies w_5: $5 = (101)_2$
$x_2 = 1\ x_1 = 0\ x_0 = 1$

Anmerkung

PLAs mit festem UND-Teil werden als PROM verkauft.

PLA als Speicher

Zwischen-Fazit PLAs

PLAs sind preiswerte, universelle Bausteine, die

- Beliebige boolesche Funktionen leicht realisierbar machen
- Minimalpolynomdarstellungen motivieren
- Speicherung von 2^n Wörtern der Länge m in einem $(n+m)*2^n$-PLA erlauben

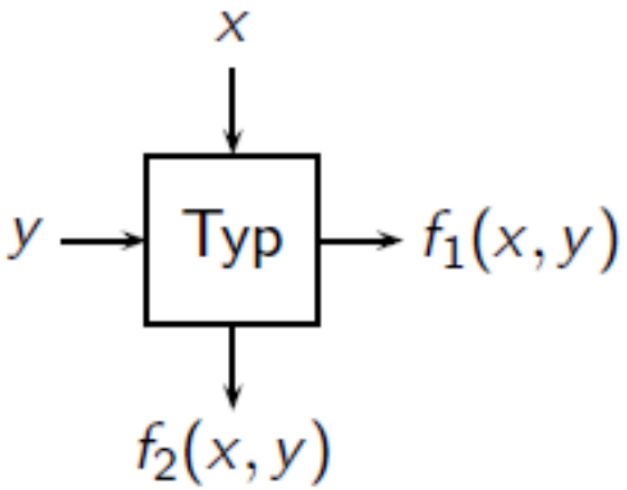
Nachteil Nur einmal programmierbar

Wie kann man PLAs beliebig neu programmierbar machen?

Programmierbare Bausteine

6. **Programmierbare Bausteine**
    1. Einleitung ✓
    2. Grundbausteine ✓
    3. Realisierung von Monomen und Polynomen ✓
    4. PLA als Speicher ✓
    5. **Software PLAs**

Software PLAs



Typ	$f_1(x,y)$	$f_2(x,y)$
0	y	x
1	$x \vee y$	x
2	y	$x\,y$
3	y	$x\,\overline{y}$

Beobachtung Vier verschiedene Typen, mit zwei Bits codierbar

- Erweitere PLA-Baustein um zwei zusätzliche Eingaben
- Diese codieren den Baustein-Typ

Lösung f_1 und f_2 als Funktionen von (s, t, x, y) darstellen

- $f_1(s,t,x,y) = y \vee \overline{s}tx$

- $f_2(s,t,x,y) = \overline{s}x \vee \overline{t}xy \vee tx\overline{y}$

Software PLAs

Vorgehensweise

1. **Nachdenken über Codierung**
2. Funktionstabelle
3. KV-Diagramm erstellen
4. KV-Diagramm bearbeiten
5. Primimplikanten aufstellen
6. Minimale Überdeckung finden
7. Schaltung realisieren

Darstellung von f_1 und f_2 als Funktionen

Typ	$f_1(x, y)$	$f_2(x, y)$
0	y	x
1	$x \vee y$	x
2	y	$x \wedge y$
3	y	$x \wedge \overline{y}$

Software PLAs

Wenn ein PLA durch eine boolesche Funktion synthetisiert werden soll, wie viele Eingaben hat die Funktion f_1 und f_2 bei geschickt gewählter Kodierung mindestens?

A: 2: x und y

B: 3: x, y und 1 Kodierungsbit

C: 4: x, y und 2 Kodierungsbits

D: 6: x, y, 4 Kodierungsbits

Software PLAs

Vorgehensweise

1. **Nachdenken über Codierung**

- Wir betrachten beide Funktionen getrennt
- 4 unterschiedliche Funktionen
  - Kodierung der Funktion mittels zwei Bits (s, t)
  - Jede Funktion hat zusätzlich zwei Eingangsvariablen (x, y)
  - f_1 und f_2 haben dann jeweils 4 Eingangsvariablen (s, t, x, y)
  - f_1 und f_2 haben dann jeweils 1 Ausgangsvariable

Darstellung von f_1 und f_2 als Funktionen

Typ	$f_1(x, y)$	$f_2(x, y)$
0	y	x
1	$x \vee y$	x
2	y	$x \wedge y$
3	y	$x \wedge \overline{y}$

Software PLAs

Vorgehensweise

1. Nachdenken über Codierung
2. **Funktionstabelle**
3. KV-Diagramm erstellen
4. KV-Diagramm bearbeiten
5. Primimplikanten aufstellen
6. Minimale Überdeckung finden
7. Schaltung realisieren

Darstellung von f_1 und f_2 als Funktionen

Typ	$f_1(x, y)$	$f_2(x, y)$
0	y	x
1	$x \vee y$	x
2	y	$x \wedge y$
3	y	$x \wedge \overline{y}$

Software PLAs

Vorgehensweise

1. Nachdenken über Codierung
2. **Funktionstabelle**

s	t	x	y	f_1
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	

s	t	x	y	f_1
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

Darstellung von f_1

Typ	$f_1(x, y)$
0	y
1	$x \vee y$
2	y
3	y

Software PLAs

Vorgehensweise

1. Nachdenken über Codierung
2. Funktionstabelle
3. **KV-Diagramm erstellen**
4. KV-Diagramm bearbeiten
5. Primimplikanten aufstellen
6. Minimale Überdeckung finden
7. Schaltung realisieren

Darstellung von f_1 und f_2 als Funktionen

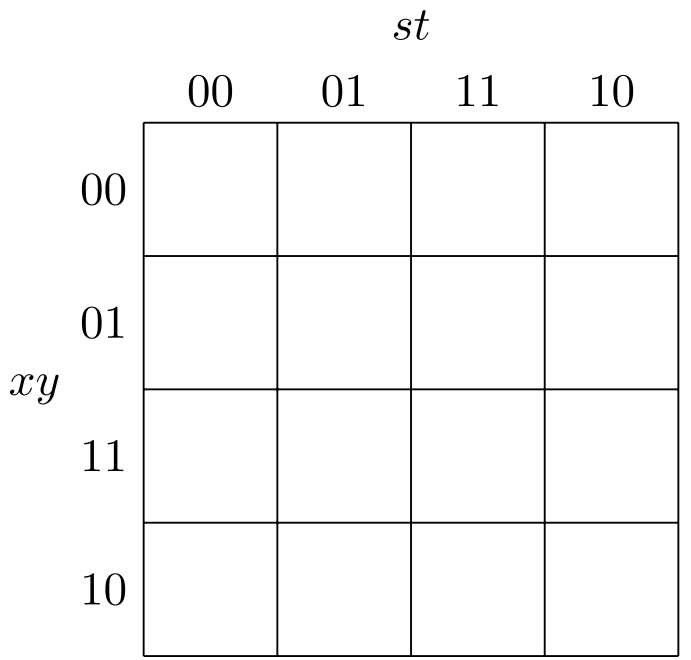
Typ	$f_1(x, y)$	$f_2(x, y)$
0	y	x
1	$x \vee y$	x
2	y	$x \wedge y$
3	y	$x \wedge \overline{y}$

Software PLAs

3. KV-Diagramm erstellen

s	t	x	y	f_1
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1

s	t	x	y	f_1
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1



Software PLAs b

3. KV-Diagramm erstellen

s	t	x	y	f_1
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1

s	t	x	y	f_1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

xy \ st	00	01	11	10
00				
01	1	1	1	1
11	1	1	1	1
10		1		

Software PLAs

Vorgehensweise

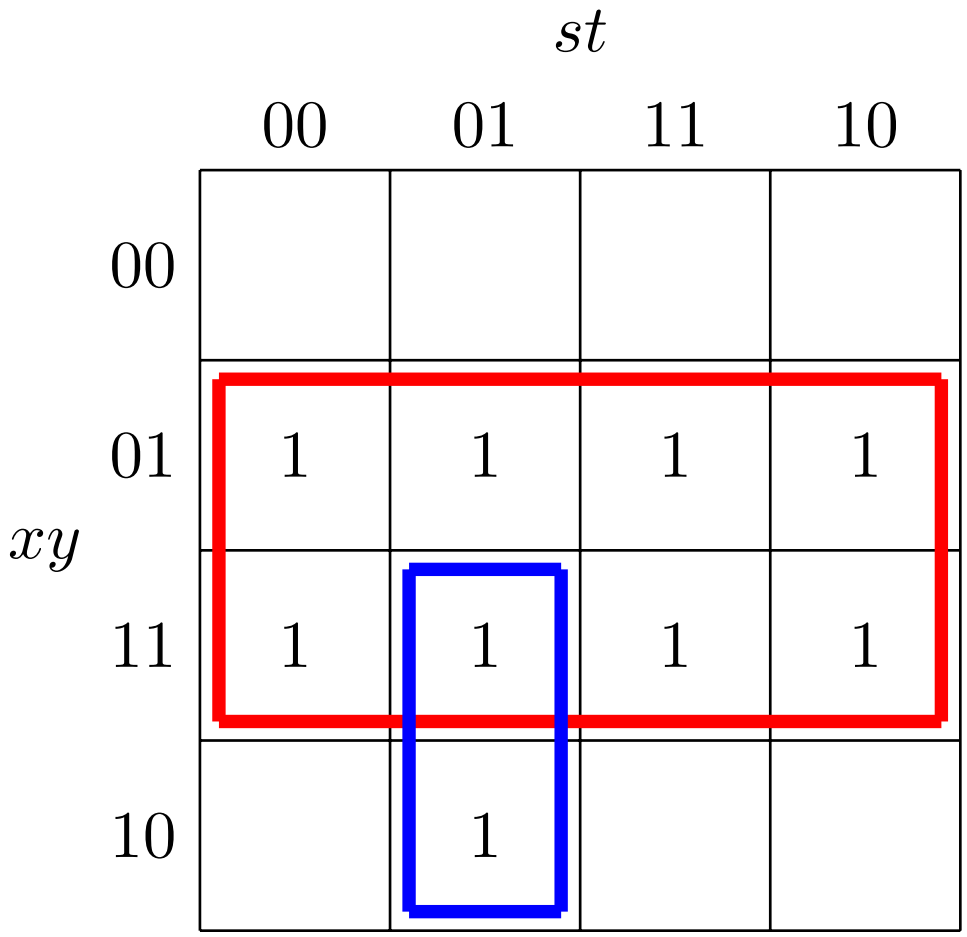
1. Nachdenken über Codierung
2. Funktionstabelle
3. KV-Diagramm erstellen
4. **KV-Diagramm bearbeiten**
5. Primimplikanten aufstellen
6. Minimale Überdeckung finden
7. Schaltung realisieren

Darstellung von f_1 und f_2 als Funktionen

Typ	$f_1(x, y)$	$f_2(x, y)$
0	y	x
1	$x \vee y$	x
2	y	$x \wedge y$
3	y	$x \wedge \overline{y}$

Software PLAs

4. **KV-Diagramm bearbeiten**



Software PLAs

Vorgehensweise

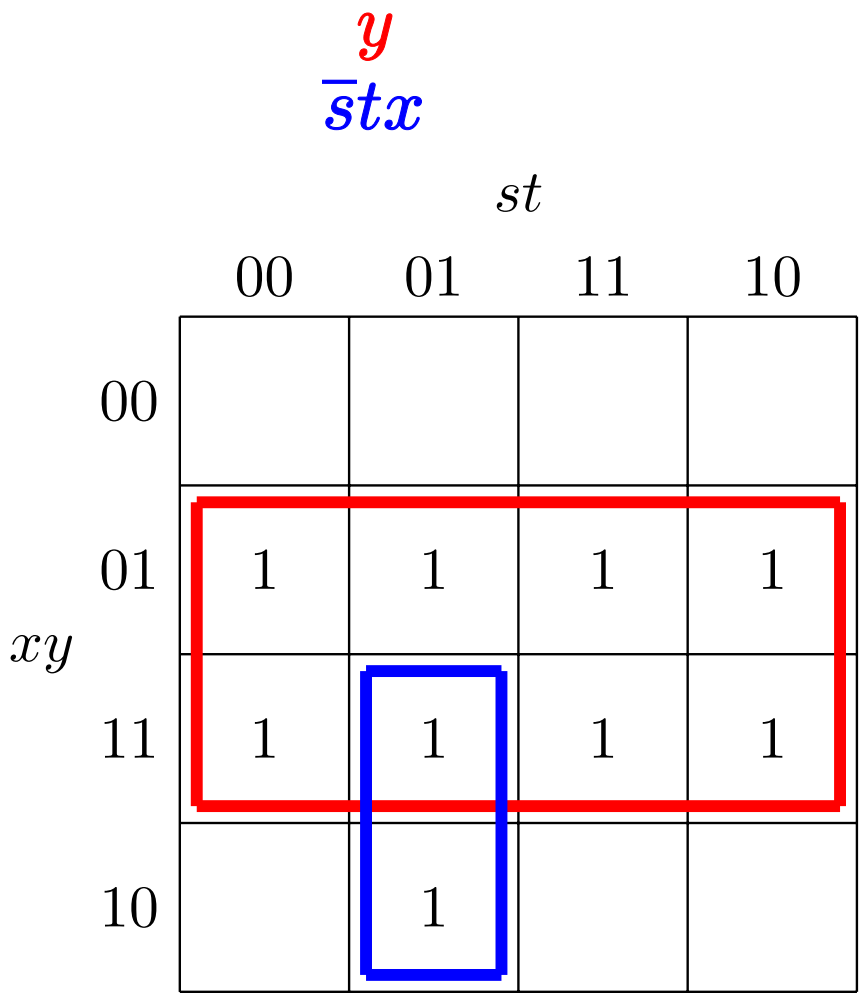
1. Nachdenken über Codierung
2. Funktionstabelle
3. KV-Diagramm erstellen
4. KV-Diagramm bearbeiten
5. **Primimplikanten aufstellen**
6. Minimale Überdeckung finden
7. Schaltung realisieren

Darstellung von f_1 und f_2 als Funktionen

Typ	$f_1(x, y)$	$f_2(x, y)$
0	y	x
1	$x \vee y$	x
2	y	$x \wedge y$
3	y	$x \wedge \overline{y}$

Software PLAs

5. **Primimplikanten aufstellen**



Software PLAs

Vorgehensweise

1. Nachdenken über Codierung
2. Funktionstabelle
3. KV-Diagramm erstellen
4. KV-Diagramm bearbeiten
5. Primimplikanten aufstellen
6. **Minimale Überdeckung finden**
7. Schaltung realisieren

Darstellung von f_1 und f_2 als Funktionen

Typ	$f_1(x, y)$	$f_2(x, y)$
0	y	x
1	$x \vee y$	x
2	y	$x \wedge y$
3	y	$x \wedge \overline{y}$

Software PLAs

6. **Minimale Überdeckung finden**

Minimale Überdeckung

$f_1(s,t,x,y) = y \vee \overline{s}tx$

Software PLAs

Vorgehensweise

1. Nachdenken über Codierung
2. Funktionstabelle
3. KV-Diagramm erstellen
4. KV-Diagramm bearbeiten
5. Primimplikanten aufstellen
6. Minimale Überdeckung finden
7. **Schaltung realisieren**

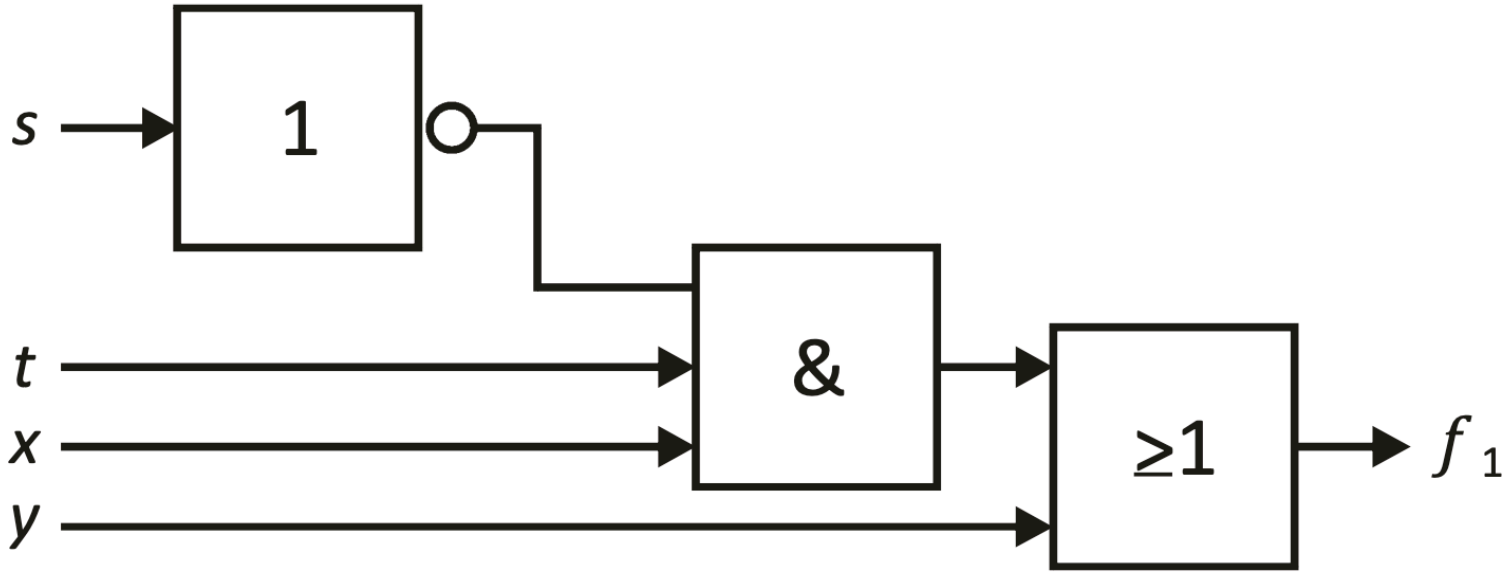
Darstellung von f_1 und f_2 als Funktionen

Typ	$f_1(x, y)$	$f_2(x, y)$
0	y	x
1	$x \vee y$	x
2	y	$x \wedge y$
3	y	$x \wedge \overline{y}$

Software PLAs

7. **Schaltung realisieren**

$f_1(s,t,x,y) = y \vee \overline{s}tx$



Darstellung von f_1 und f_2 als Funktionen

Typ	$f_1(x,y)$	$f_2(x,y)$
0	y	x
1	$x \vee y$	x
2	y	$x \wedge y$
3	y	$x \wedge \overline{y}$

Software PLAs

Vorgehensweise analog für f2

1. Nachdenken über Codierung
2. Funktionstabelle
3. KV-Diagramm erstellen
4. KV-Diagramm bearbeiten
5. Primimplikanten aufstellen
6. Minimale Überdeckung finden
7. Schaltung realisieren

Darstellung von f_1 und f_2 als Funktionen

Typ	$f_1(x, y)$	$f_2(x, y)$
0	y	x
1	$x \vee y$	x
2	y	$x \wedge y$
3	y	$x \wedge \overline{y}$

Software PLAs

Bemerkung zum Einsatz

Beobachtung

- Für ein $n * x$-PLA werden zur Programmierung
- .. 2 *nm* Bits gebraucht
- Man kann diese 2 *nm* Bits gut in einem PROM speichern

Fazit

- Einfache und günstige Realisierung von booleschen Funktionen
- Besonders geeignet für kleine Stückzahlen
- Besonders geeignet bei nur temporärem Gebrauch

Software PLAs

Varianten von PLAs

- PAL (Programmable Array Logic)
  - PLA
  - Nur UND-Array ist programmierbar
  - Einmal programmierbar
- GAL (Generic Array Logic)
  - PLA
  - Nur UND-Array ist programmierbar
  - Löschbar (UV-Licht oder elektrisch)
- CPLD (Complex Programmable Logic Device)
  - Programmierbare AND/OR-Matrix
  - Programmierbare Rückkopplung
  - Ein-/Ausgabeblöcke aus Registern
- FPGA (Fiel Programmable Gate Array)
  - Ähnlich CPLD, aber beliebige Funktionen im Grundbaustein realisierbar

Programmierbare Bausteine

6. **Programmierbare Bausteine**
    1. Einleitung ✓
    2. Grundbausteine ✓
    3. Realisierung von Monomen und Polynomen ✓
    4. PLA als Speicher ✓
    5. Software PLAs ✓